

Budger — Stretch Feature

GitHub Push Plan · export/backend & export/frontend

Summary

The stretch feature is fully implemented in the local dev workspace (artifacts/) but does not yet exist anywhere in the export/ directories that power the live app. This plan covers all 13 files that need to be created or modified to ship it.

What stretch does:

- Cross-category stretch — reallocates budget from one category to another within the same month.
- Cross-month stretch — borrows from the same category's next-month budget (capped at 50%, 2-month cooldown).
- Managed from the Categories tab via a dedicated dialog; not from the transaction add/edit form.

Deploy order: Migration runs automatically on Render startup before any request hits the new route. Backend and frontend can be pushed in a single commit.

Files at a Glance

#	File	Type	Scope
1	export/backend/migrations/0009_budget_stretches.sql	NEW	DB table creation
2	export/backend/src/db/schema/budget_stretches.ts	NEW	Drizzle schema
3	export/backend/src/db/schema/index.ts	EDIT	+1 export line
4	export/backend/src/routes/budget-stretches.ts	NEW	GET / POST / PATCH / DELETE
5	export/backend/src/routes/index.ts	EDIT	+2 lines
6	.../api-client/generated/api.schemas.ts	EDIT	+BudgetStretch types
7	.../api-client/generated/api.ts	EDIT	+4 hook groups appended
8	export/frontend/src/pages/Categories.tsx	EDIT	Full stretch feature
9	.../components/DonutBudgetChart.tsx	EDIT	Orange stretch segments
10	export/frontend/src/pages/Dashboard.tsx	EDIT	Adjusted budget + orange label
11	.../components/HouseholdDonutChart.tsx	EDIT	Orange ring for stretched users
12	export/frontend/src/pages/Household.tsx	EDIT	Stretch indicator + adjusted budget
13	export/frontend/src/pages/HomeSpending.tsx	EDIT	MINIMAL — orange label + headline

1 — Database Migration

File: export/backend/migrations/0009_budget_stretches.sql (NEW)

One clean migration combining local 0007 + 0008. Creates the table with transaction_id already nullable (the export DB has never had this table, so there is no need for a two-step ALTER).

```
CREATE TABLE "budget_stretches" (  
  "id" serial PRIMARY KEY NOT NULL,  
  "user_id" integer NOT NULL,
```

```

"transaction_id" integer, -- nullable
"month" text NOT NULL,
"to_category_id" integer NOT NULL,
"from_category_id" integer NOT NULL,
"amount" numeric(12, 2) NOT NULL,
"stretch_type" text NOT NULL,
"created_at" timestamp with time zone DEFAULT now() NOT NULL,
CONSTRAINT "budget_stretches_transaction_id_unique" UNIQUE("transaction_id")
);
ALTER TABLE "budget_stretches"
ADD CONSTRAINT "budget_stretches_user_id_fk"
FOREIGN KEY ("user_id") REFERENCES "public"."users"("id") ON DELETE cascade;
ALTER TABLE "budget_stretches"
ADD CONSTRAINT "budget_stretches_to_cat_fk"
FOREIGN KEY ("to_category_id") REFERENCES "public"."categories"("id") ON DELETE cascade;
ALTER TABLE "budget_stretches"
ADD CONSTRAINT "budget_stretches_from_cat_fk"
FOREIGN KEY ("from_category_id") REFERENCES "public"."categories"("id") ON DELETE cascade;
CREATE INDEX "budget_stretches_user_id_month_idx"
ON "budget_stretches" USING btree ("user_id", "month");
CREATE INDEX "budget_stretches_to_category_id_month_idx"
ON "budget_stretches" USING btree ("to_category_id", "month");

```

The Drizzle migrations journal (`migrations/meta/_journal.json`) must also receive a new entry with the SHA-256 hash of the migration file content, matching the format of existing entries.

2 — Backend Schema: `budget_stretches.ts` (NEW)

File: `export/backend/src/db/schema/budget_stretches.ts`

Direct copy from `lib/db/src/schema/budget_stretches.ts`. Relative imports (`./users`, `./categories`) are identical to the existing export schema folder structure — no path changes needed.

Defines:

- `budgetStretchesTable` — `pgTable` with all columns, indexes, and constraints.
- `insertBudgetStretchSchema` — Zod insert schema via `drizzle-zod`.
- `InsertBudgetStretch` and `BudgetStretch` TypeScript types.

3 — Backend Schema Index (EDIT)

File: `export/backend/src/db/schema/index.ts` — add one line:

```
export * from "./budget_stretches";
```

4 — Backend Route: `budget-stretches.ts` (NEW)

File: `export/backend/src/routes/budget-stretches.ts`

Implements four endpoints:

Method	Path	Description
GET	<code>/budget-stretches?month=YYYY-MM</code>	List stretches for the authenticated user in a given month
POST	<code>/budget-stretches</code>	Create a stretch (validates ownership, 50% cap, 2-month cooldown for <code>cross_month</code>)
PATCH	<code>/budget-stretches/:id</code>	Update amount only — called by the edit dialog; re-validates 50% cap

DELETE	/budget-stretches/:id	Remove a stretch (ownership check)
--------	-----------------------	------------------------------------

5 — Backend Route Index (EDIT)

File: export/backend/src/routes/index.ts — add import + router.use:

```
import budgetStretchesRouter from "../budget-stretches";
// ... existing routers ...
router.use(budgetStretchesRouter);
```

6 — API Client Schemas (EDIT)

File: export/frontend/src/lib/api-client/generated/api.schemas.ts

Append to end of file:

- BudgetStretchStretchType const + union type ('cross_category' | 'cross_month')
- BudgetStretch interface — id, userId, transactionId, month, toCategoryId, fromCategoryId, amount, stretchType, createdAt
- BudgetStretchInput interface — same fields minus id/createdAt (transactionId optional)
- ListBudgetStretchesParams type — { month: string }
- Also verify: CategorySpending type may need isStretched / stretchAmount fields if DonutBudgetChart passes them via the summary endpoint

7 — API Client Hooks (EDIT)

File: export/frontend/src/lib/api-client/generated/api.ts

Append all budget-stretches hooks at end of file (ported from lib/api-client-react/src/generated/api.ts lines ~6892+):

- getListBudgetStretchesUrl / listBudgetStretches / getListBudgetStretchesQueryKey / getListBudgetStretchesQueryOptions / useListBudgetStretches
- getCreateBudgetStretchUrl / createBudgetStretch / getCreateBudgetStretchMutationOptions / useCreateBudgetStretch
- getUpdateBudgetStretchUrl / updateBudgetStretch / getUpdateBudgetStretchMutationOptions / useUpdateBudgetStretch
- getDeleteBudgetStretchUrl / deleteBudgetStretch / getDeleteBudgetStretchMutationOptions / useDeleteBudgetStretch

No change needed to api-client/index.ts — it already re-exports everything from ./generated/api and ./generated/api.schemas.

8 — Categories.tsx (EDIT — largest change)

File: export/frontend/src/pages/Categories.tsx

Import additions:

```
import { useListBudgetStretches, getListBudgetStretchesQueryKey } from "@lib/api-client";
import { ArrowRightLeft } from "lucide-react"; // added to existing lucide import line
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@components/ui/select";
```

New component: StretchCategoryDialog (~230 lines)

- Props: category, categories, existingStretch, open, onClose, receiverCatIds
- State: isCrossMonth toggle, toCatId (donor category), amount, saving, error
- Edit mode (existingStretch != null): PATCH on submit, DELETE when amount set to 0
- Create mode: POST with stretchType from isCrossMonth toggle
- Cross-month cap: 50% of category.budget, displayed inline
- Orange dialog with ArrowRightLeft icon; disabled while saving

CategoryCard: extended props + new UI

- New props: stretchInfo, allCategories, existingStretch, isDonor, receiverCatIds
- $\text{effectiveBudget} = \text{Math.max}(0, \text{base} + \text{stretchInfo.toAmt} - \text{stretchInfo.fromAmt})$
- Progress bar width + label use effectiveBudget instead of raw category.budget
- Orange inline label: (+€XX stretch) when toAmt > 0
- Lighter orange inline label: (–€XX donated) when fromAmt > 0
- Orange 'Stretch' button at bottom; disabled with tooltip when isDonor
- Renders when stretchOpen state is true

Main component additions:

- useListBudgetStretches({ month: currentMonth }) hook call
- stretchByCatId Map — keyed by toCategoryId and fromCategoryId
- stretchObjectsByCatId Map — raw stretch object per category (for edit dialog)
- donorCatIds Set — categories acting as donors (their Stretch button is disabled)
- CategoryCard render call updated to pass all new props

9 — DonutBudgetChart.tsx (EDIT)

File: export/frontend/src/components/DonutBudgetChart.tsx

- New Props field: adjustedTotalBudget?: number | null
- Category segment input type gains: isStretched?: boolean, stretchAmount?: number, stretchType?: string | null
- $\text{effectiveTotalBudget} = \text{adjustedTotalBudget} ?? \text{totalBudget}$ — used for center percentage calculation
- Segment border: orange (#c47a2a, strokeWidth 1.5) when isStretched && !isOverBudget
- Legend: stretched category percentage rendered in orange (#f97316) instead of muted-foreground
- Center donut %: computed against effectiveTotalBudget (reflects cross-month borrow)

10 — Dashboard.tsx (EDIT)

File: export/frontend/src/pages/Dashboard.tsx

- Add useListBudgetStretches to @/lib/api-client import
- Hook call: useListBudgetStretches({ month: viewMonth })
- Compute crossMonthNetAmt — sum of cross_month stretch amounts
- $\text{adjustedTotalBudgetDash} = \text{totalBudget} + \text{crossMonthNetAmt}$ (when non-zero)
- adjustedTotalBudgetForChart — same but based on totalBudgetForChart (chart's budget denominator)
- Budget stat card: percentage + BudgetBar use adjustedTotalBudgetDash ?? totalBudget
- If adjustedTotalBudgetDash != null: show orange '+€XX from next month' label

- DonutBudgetChart: add `adjustedTotalBudget={adjustedTotalBudgetForChart}` prop

11 — HouseholdDonutChart.tsx (EDIT)

File: `export/frontend/src/components/HouseholdDonutChart.tsx`

- New prop: `crossMonthStretchUserIds?: number[]`
- `borderPath` function: receives `isStretched` boolean; uses `#c47a2a` stroke + `strokeWidth 1.5` when true
- Group segment rendering: passes `crossMonthStretchUserIds?.includes(gb.userId)` to `borderPath`

12 — Household.tsx (EDIT)

File: `export/frontend/src/pages/Household.tsx`

- Add `useListBudgetStretches` to `@/lib/api-client` import
- Hook: `useListBudgetStretches({ month: hhCurrentMonth })`
- `hasCrossMonthStretch` — boolean, true if any `cross_month` stretch exists this month
- `crossMonthStretchUserIds` — `[me.id]` when `hasCrossMonthStretch`, else `[]`
- `myStretchAmountRaw` — sum of `cross_month` stretch amounts
- Member list: for current user, compute `adjustedBudget = inViewerCurrency + stretchInViewerCurrency` (currency-converted)
- `isStretched` flag: show `adjustedBudget` in `text-orange-500` when `> 0`
- `HouseholdDonutChart` call: add `crossMonthStretchUserIds={crossMonthStretchUserIds}` prop

13 — HomeSpending.tsx (EDIT — MINIMAL)

File: `export/frontend/src/pages/HomeSpending.tsx`

■ **Only two visible UI changes. No changes to TxForm or the add/edit transaction windows.**

Visible change 1 — Adjusted budget headline:

```
<AmtHero amount={adjustedTotalBudget ?? totalBudget} currency={prefs.currency} />
```

Visible change 2 — Orange stretch label below headline:

```
{adjustedTotalBudget !== null && (
  <p className="text-xs text-orange-400 mt-0.5">
    +<AmtHero amount={crossMonthNetAmt} />
    {prefs.language === 'pl' ? 'przeniesione z nast. miesi■ca' : 'stretched from next month'}
  </p>
)}
```

Supporting code additions (not visible on their own):

- `useListBudgetStretches` hook call (needed to power the two display changes above)
- `stretchByTxId` Map — maps `transactionId` → stretch (computed alongside existing maps)
- `crossMonthNetAmt` calculation — sum of `cross_month` amounts
- `adjustedTotalBudget = totalBudget + crossMonthNetAmt` when non-zero
- `hasStretch` variable added per transaction row; included in existing `badge-container` condition — no new visible badge span rendered

Import path note: all local artifacts use @workspace/api-client-react; all export/frontend files use @/lib/api-client — this path difference is applied throughout.

PATCH /budget-stretches/:id is implemented in the backend route but not in the OpenAPI spec. Categories.tsx calls apiFetch directly for updates — no generated hook needed.